

CS101: Computer Programming



Lec 01: Introduction to Computing

Dr. Soumitra Ghosh
Assistant Professor, CSE
IIT (BHU) Varanasi

soumitra.cse@iitbhu.ac.in

How the courses are run? Policies, Guidelines..

- There are 8 TAs who will primarily help you in the Laboratory sessions and Tutorial Classes.
- **Questions in the examinations** will be from what will be taught in the classes.
- Ask your doubts and questions to me. Do not discuss in the class with your friends.
- **Attendance for Laboratory sessions is important** and will be counted.
- **Grading** will be strictly according to **your performance, attendance**, etc.

Course Objectives

- To give basic programming skills.
- To give conceptual understanding of computers and their working.
- At the end of the course the student is expected to write a moderately complex
- C program.

Evaluation Criteria

Theory

- Mid Semester Exam: 30%
- End Semester Exam: 40%
- Assignments / Quizzes: 10%

Laboratory:

- Assignments + Viva: 20%

Laboratory Part

- Use only *Linux/Ubuntu/Fedora* Operating System.
[Do not use *Microsoft windows* !]
- Use ANSI C (Standard C).
- Use *vi/nano* editor for creating your files.
- Brief about the 1st exercise will be given towards the end of this class.

Reference Books and Course Page

You may follow any standard ANSI C book.

- C Programming: A Modern Approach by K. N. King
- Let us C, Yashavant Kanetkar
- **Programming in ANSI C, E. Balagurusamy**
- Programming With C pdf – Byron Gottfried
- Brian W. Kernighan, Dennis M. Ritchie. The C Programming Language

Initially Lecture slides and materials will be shared via Gdrive

In due time, a dedicated course page will be designed for materials dissemination.

Learning Objectives

In this lecture you will learn about:

- Computer
- Characteristic features of computers
- Stored Program Concept
 - von Neumann bottleneck
 - Language Evolution
 - Compiler - Interpreter

Computer

- The word computer comes from the word “**compute**”, which means, “to calculate”
- Thereby, a computer is an electronic device that can perform arithmetic operations at high speed
- A computer is also called a *data processor* because it can store, process, and retrieve data whenever desired

Computer

- The word computer comes from the word “**compute**”, which means, “to calculate”
- Thereby, a computer is an electronic device that can perform arithmetic operations at high speed
- A computer is also called a *data processor* because it can store, process, and retrieve data whenever desired

Characteristics

? ? ? ? ? ? ? ?

Computer

- The word computer comes from the word “**compute**”, which means, “to calculate”
- Thereby, a computer is an electronic device that can perform arithmetic operations at high speed
- A computer is also called a *data processor* because it can store, process, and retrieve data whenever desired

Characteristics

Automatic | Speed | Accuracy | Diligence | No IQ | No Feelings | Versatility | Power of Remembering

Stored Program Concept

Key Idea: Both the program instructions and the data are stored together in the computer's main memory.

- CPU fetches instructions one by one from memory, decodes them, executes them, and stores the results back in memory.
- Allows:
 - Easy modification of programs
 - Reusability
 - Automatic execution without rewiring hardware

Before Stored Program ?????

Before Stored Program

Programs were:

- Hard-wired, connected using switches and cables
- Difficult to modify and time-consuming to reprogram
- Every new task required rewiring, restructuring and/or redesigning the fixed-program machine
- Also known as **Fixed Programs**
 - *Example: Calculator*
 - Can do basic mathematics, but cannot be used as word processor or gaming console
 - *ENIAC*: It could take 3 weeks to set up a program and get it working

Technicians connecting wires of ENIAC

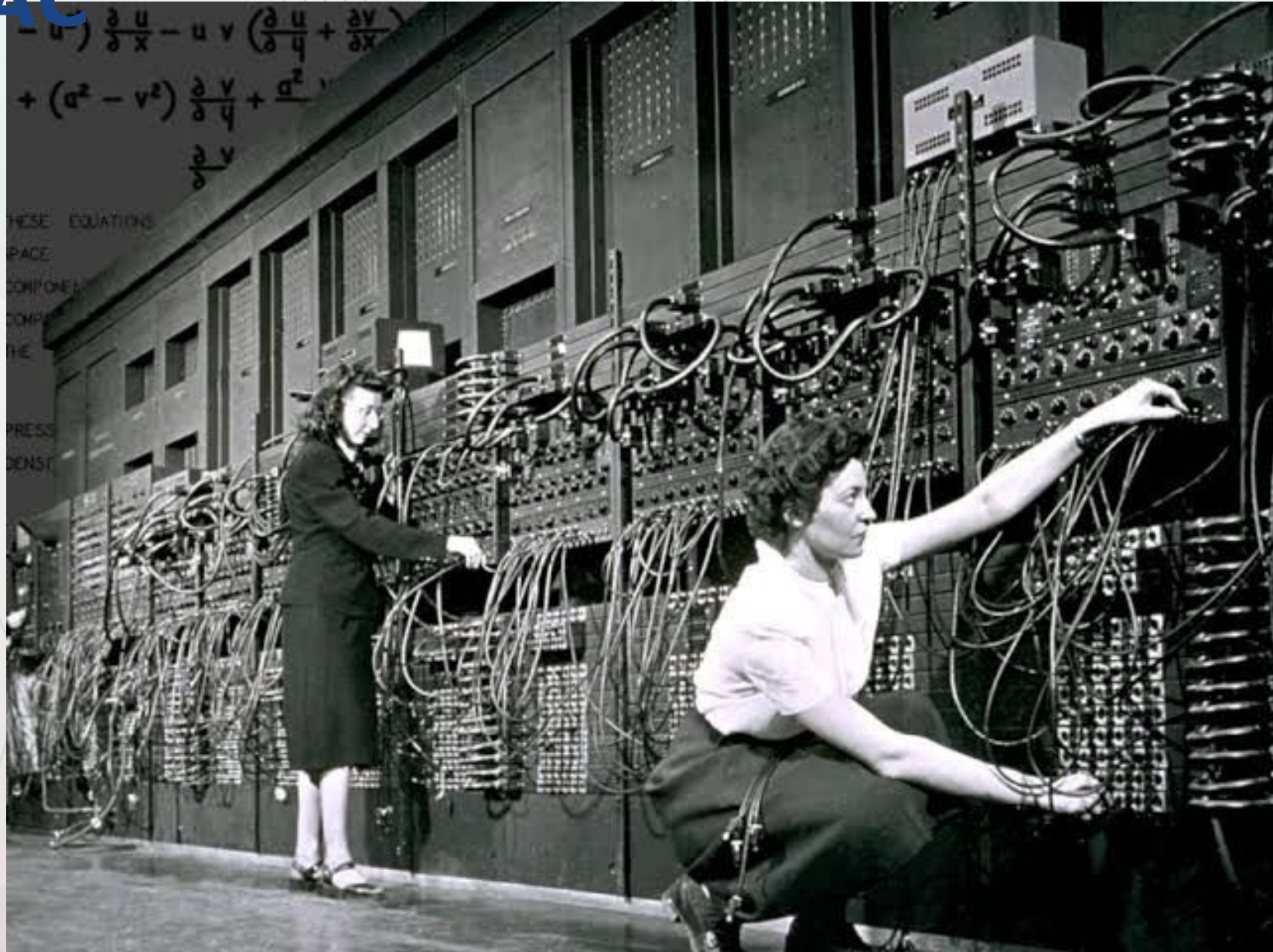


Photo source: <https://cse.engin.umich.edu/about/history/eniac-display/>

History of the Stored Program Concept

★ Early Computers (1940s)

- ENIAC, Colossus
- Characteristics:
 - No stored program; Programs entered manually; Wiring changes required for every task

★ John von Neumann (1945)

- Proposed the Stored Program Architecture in the report:
 - *First Draft of a Report on the EDVAC*
 - Main idea:
 - Store instructions and data together in memory.
 - CPU executes instructions sequentially.

★ First Stored Program Computers

- Manchester Baby (1948), EDSAC (1949), EDVAC, IAS Computer
- These became the basis of modern computers.

First Draft of a Report
on the EDVAC

by

John von Neumann



(John) von Neumann

also known as *Princeton Architecture*

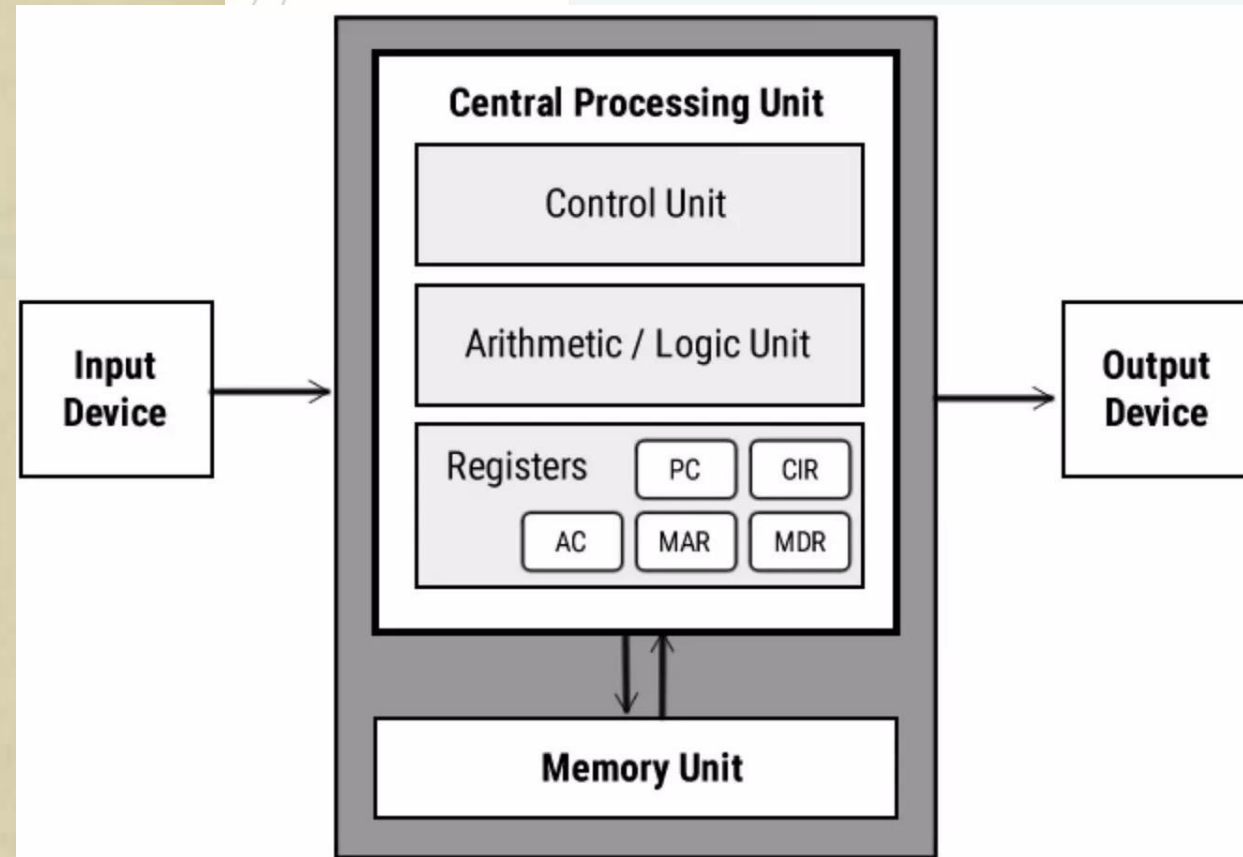
Architecture

Contract No. W-670-ORD-4926

Between the
United States Army Ordnance Department
and the
University of Pennsylvania

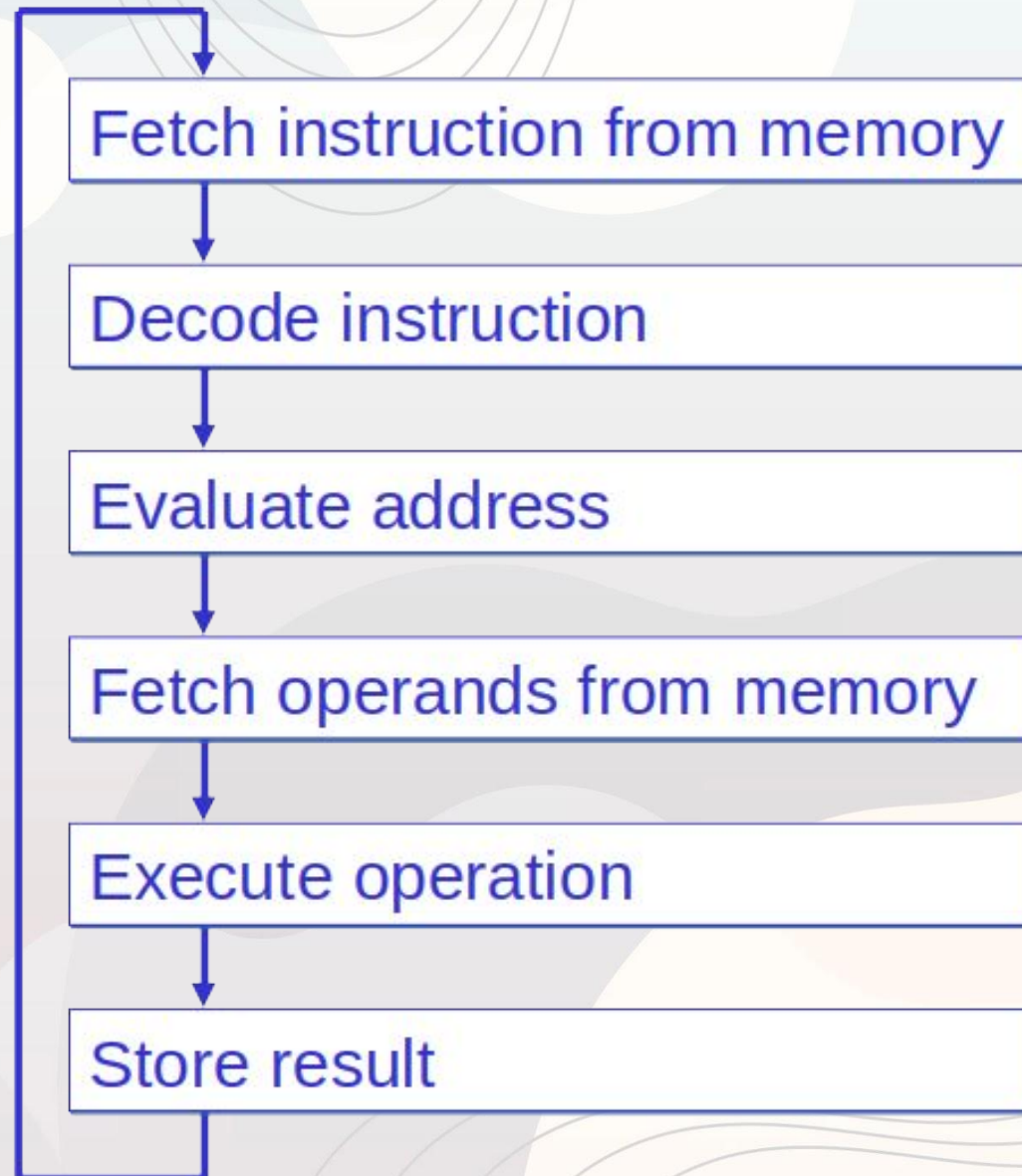
Moore School of Electrical Engineering
University of Pennsylvania

June 30, 1945



Working of Von Neumann Model

Instruction Processing



Stored Program Concept: Advantages & Disadvantages

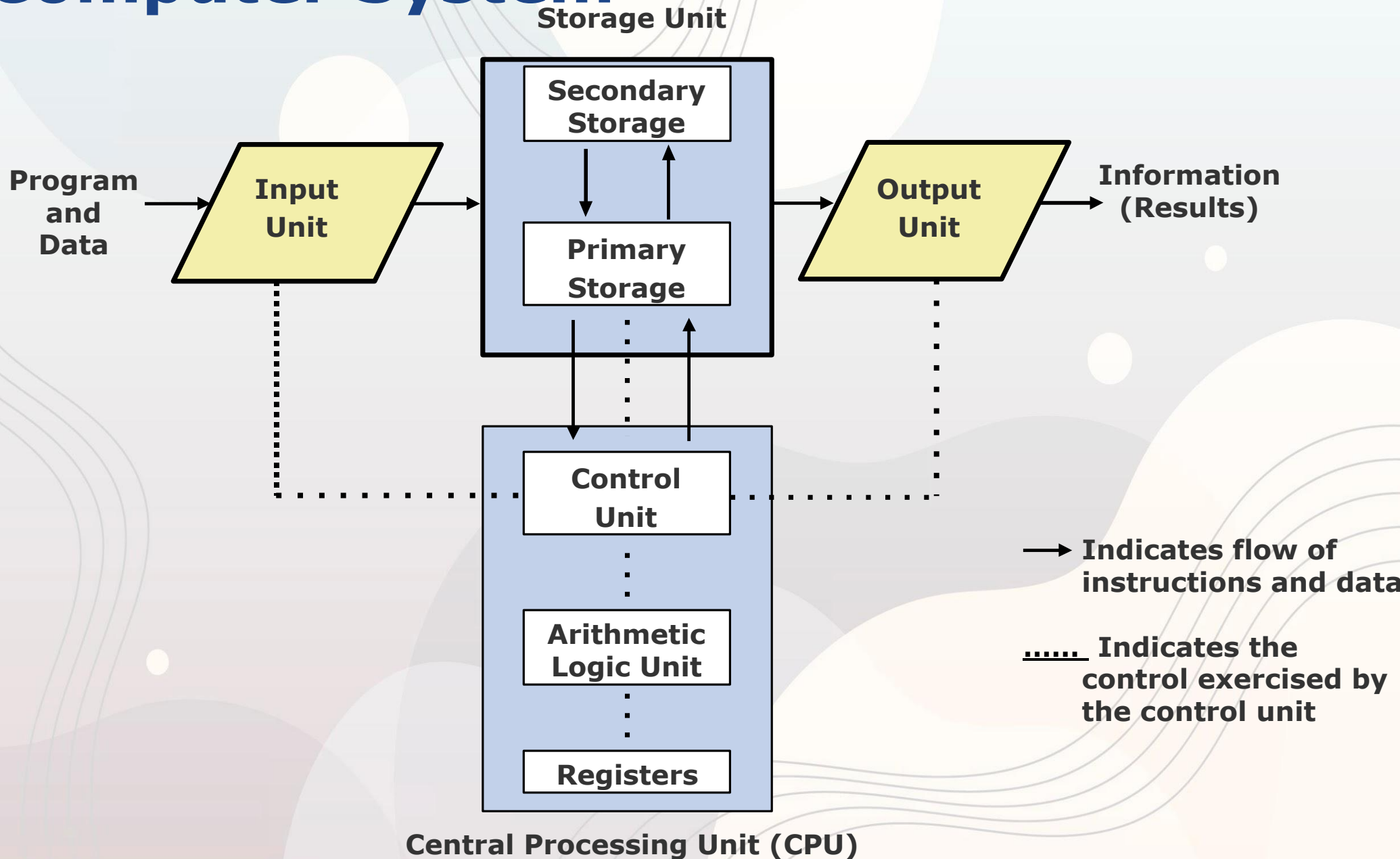
Advantages

- ✓ Programs and data can be stored together in memory.
- ✓ Easy to modify or update programs without changing hardware.
- ✓ Supports automatic execution using the Fetch–Decode–Execute cycle.
- ✓ Same hardware can run different applications by simply changing the program.
- ✓ Simplifies programming and enables general-purpose computing.

Disadvantages

- ✗ Instructions and data share the same memory, causing contention.
- ✗ Suffers from the Von Neumann Bottleneck due to limited CPU–memory bandwidth.
- ✗ Sequential instruction execution limits parallelism.
- ✗ Performance depends heavily on memory access speed.
- ✗ Not ideal for modern AI and data-intensive applications requiring massive parallel processing.

Basic Organization of a Computer System



The Five Basic Operations of a Computer System

- **Inputting.** The process of entering data and instructions into the computer system
- **Storing.** Saving data and instructions to make them readily available for initial or additional processing whenever required
- **Processing.** Performing arithmetic operations (add, subtract, multiply, divide, etc.) or logical operations (comparisons like equal to, less than, greater than, etc.) on data to convert them into useful information
- **Outputting.** The process of producing useful information or results for the user such as a printed report or visual display
- **Controlling.** Directing the manner and sequence in which all of the above operations are performed

Input Unit

1. Accepts (or reads) instructions and data from outside world
2. Converts these instructions and data in computer acceptable form
3. Supplies the converted instructions and data to the computer system for further processing

Output Unit

1. Accepts the results produced by the computer, which are in coded form and hence, cannot be easily understood by us
2. Converts these coded results to human acceptable (readable) form
3. Supplies the converted results to outside world

Storage Unit

Holds (or stores) the following :

1. Data and instructions required for processing (received from input devices)
2. Intermediate results of processing
3. Final results of processing, before they are released to an output device

Primary storage

- Hold running program instructions
- Hold data, intermediate results, and results of ongoing job(s)
- Fast in operation
- Small Capacity
- Expensive and Volatile

Secondary storage

- Hold stored program instructions
- Hold data and information of stored jobs
- Slower than primary storage
- Large Capacity
- Lot cheaper than primary storage
- Retains data even without power

Central Processing Unit (CPU)

Place where the actual executions of instructions takes place during processing operation

Manages and coordinates the operations of all other components of the computer system

**Arithmetic
Logic Unit
(ALU)**

+

**Control
Unit (CU)**

=

**Central
Processing
Unit (CPU)**

- It is the brain of a computer system
- It is responsible for controlling the operations of all other units of a computer system

CPU Registers

Register	Purpose	Example
AC Accumulator Register	Stores intermediate arithmetic and logical results produced by the ALU.	$10 + 20 = 30 \rightarrow$ Result stored in AC
MAR Memory Address Register	Holds the address of the memory location to be read from or written to.	To read data from address 2050, MAR = 2050
MDR Memory Data Register	Holds the data being transferred between CPU and main memory.	Data "75" read from memory is stored in MDR
PC Program Counter	Stores the address of the next instruction to be executed. Updated after every instruction.	If current instruction is at 1000, PC may point to 1004 (assuming 4-byte instructions)
CIR Current Instruction Register (also called Instruction Register - IR)	Holds the instruction currently being decoded and executed by the CPU.	Stores instruction ADD R1, R2 during execution

John von Neumann **Bottleneck**

The **limited data transfer speed** between the CPU and memory is called the Von Neumann Bottleneck.

Why does it occur?

- CPU speed $\gggg$ Memory speed
- Only one bus is available.
- Instructions and data compete for access.

Consequences

- CPU idle time and increased execution time
- Lower throughput and reduced performance

John von Neumann **Bottleneck**

The **limited data transfer speed** between the CPU and memory is called the Von Neumann Bottleneck.

Why does it occur?

- CPU speed $\gggg$ Memory speed
- Only one bus is available.
- Instructions and data compete for access.

Consequences

- CPU idle time and increased execution time
- Lower throughput and reduced performance

**Overcoming
the Von
Neumann
Bottleneck**



??

John von Neumann **Bottleneck**

The limited data transfer speed between the CPU and memory is called the Von Neumann Bottleneck.

Why does it occur?

- CPU speed $\gggg$ Memory speed
- Only one bus is available.
- Instructions and data compete for access.

Consequences

- CPU idle time and increased execution time
- Lower throughput and reduced performance

Overcoming the Von Neumann Bottleneck

Improving Von Neumann

Cache Memory
Pipelining
Superscalar Execution
Branch Prediction
Multi-core Processors

Beyond Von Neumann

Harvard Architecture
Dataflow Architecture
Neuromorphic Computing
Processing-in-Memory
Quantum Computing